Deep Reinforcement Learning (Q-Learning)

Internship Project Report — Task 3

1. Introduction

Reinforcement learning (RL) is a paradigm in which an agent learns to make decisions by interacting with an environment and receiving feedback in the form of rewards. Unlike supervised learning, there are no labeled examples to imitate; instead, the agent must discover a good strategy through trial and error. This project implements Q-learning, one of the foundational RL algorithms, to train an agent that navigates the FrozenLake-v1 environment from the Gymnasium library.

2. Objective

The objective is to train an agent to learn the optimal action to take in every state of the environment, purely from reward signals, using a tabular Q-learning approach, and to evaluate how well the resulting policy performs.

3. Environment: FrozenLake-v1

FrozenLake-v1 is a 4x4 grid-world environment provided by Gymnasium. The agent starts at a fixed Start tile and must reach a Goal tile without falling into any Hole tiles.

```
S F F F
F H F H
F F F H
H F F G
```

The agent has 4 discrete actions (LEFT, DOWN, RIGHT, UP) and 16 discrete states. Reaching the Goal yields a reward of 1; falling into a Hole ends the episode with reward 0; every other step yields 0 reward. The environment was run with is_slippery=True, meaning the chosen action succeeds only part of the time and the agent may slide in an unintended direction — this stochasticity makes the problem a genuine reinforcement learning challenge rather than a deterministic maze.

4. Workflow

```
Agent → Environment → Reward → Update Q-Table → Learn Policy
```

5. Methodology — Q-Learning Algorithm

The agent maintains a Q-table of shape (16 states x 4 actions), initialized to zero. After every step taken in the environment, the Q-table is updated using the Bellman equation:

```
Q(s, a) ← Q(s, a) + alpha * [ r + gamma * max_a' Q(s', a') - Q(s, a) ]
```

During training, the agent follows an epsilon-greedy policy: with probability epsilon it takes a random action (exploration), and otherwise takes the action with the highest known Q-value (exploitation).

Epsilon starts at 1.0 and decays exponentially toward a minimum of 0.01 over the course of training, so the agent explores heavily at first and increasingly relies on its learned knowledge as training progresses.

Hyperparameter	Value
Episodes	15,000
Max steps per episode	100
Learning rate (alpha)	0.1
Discount factor (gamma)	0.99
Epsilon start / min	1.0 / 0.01
Epsilon decay rate	0.0005

6. Results

During training, the agent's total reward across all 15,000 episodes was 7,052, giving an overall success rate of 47.0%. Because early episodes are dominated by random exploration, the success rate in the final 1,000 training episodes (66.5%) is a better indicator of the learned policy's quality. After training, the frozen greedy policy (always choosing the action with the highest Q-value, with no further exploration) was evaluated over 1,000 fresh test episodes.

Metric	Value
Training — overall success rate	47.0%
Training — success rate (last 1000 episodes)	66.5%
Evaluation — success rate (greedy policy)	74.1%
Evaluation — average episode length	44.3 steps

The evaluation success rate (74.1%) exceeding the late-training success rate (66.5%) is expected, since evaluation uses the purely greedy policy with zero exploration, while even late-stage training still retains a small residual epsilon causing occasional random moves. Both figures are far above the near-0% success rate of a random policy on this slippery variant of the environment, confirming that meaningful learning took place.

7. Visualizations

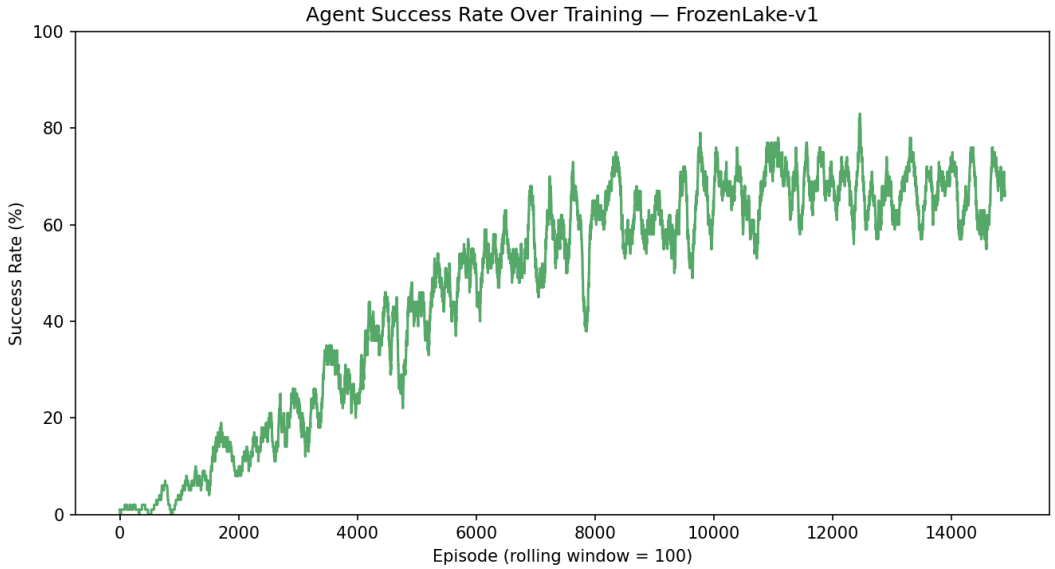

Figure 1: Rolling success rate (100-episode window) during training.

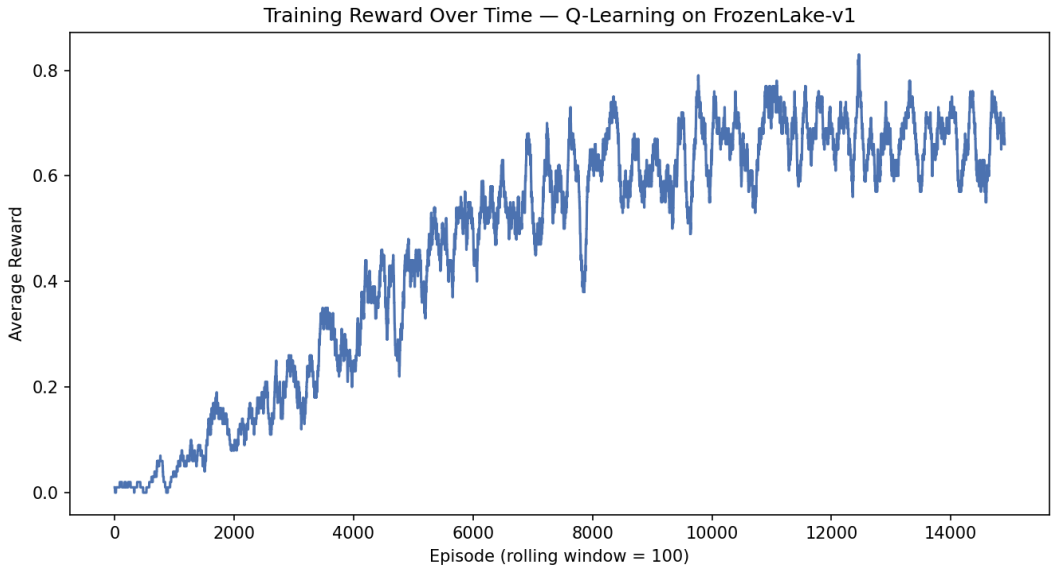

Figure 2: Rolling average reward (100-episode window) during training.

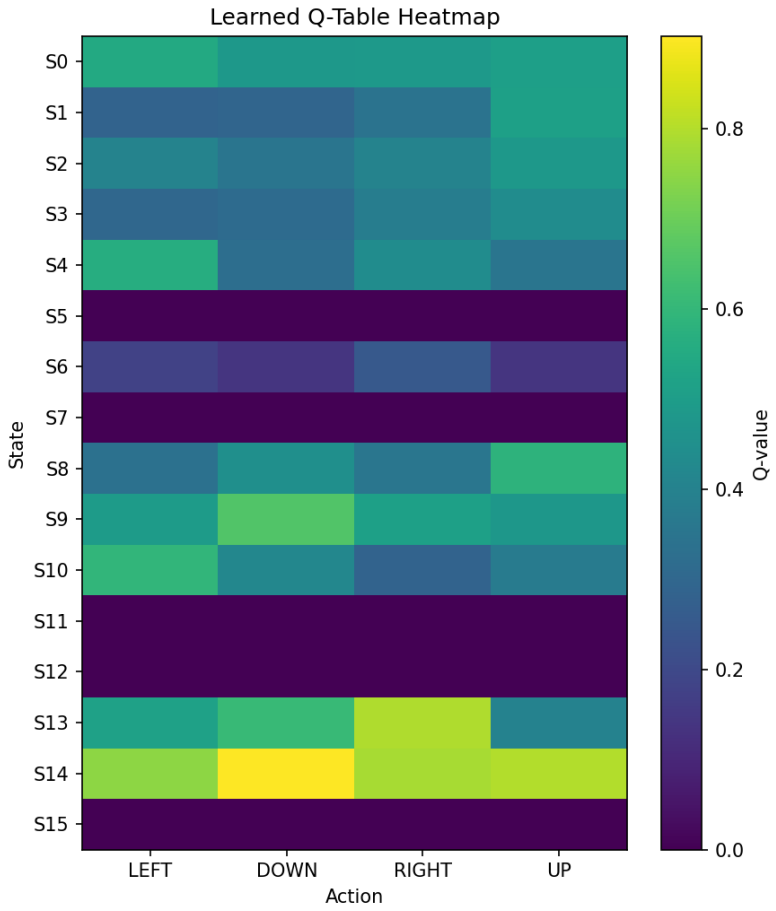

Figure 3: Heatmap of the final learned Q-table (16 states x 4 actions).

8. Conclusion

This project demonstrates a complete tabular Q-learning pipeline: environment setup, Q-table initialization, epsilon-greedy training with Bellman updates, and rigorous evaluation of the resulting policy. Despite the stochastic nature of the slippery FrozenLake-v1 environment, the agent learned a policy that reaches the goal roughly 74% of the time under greedy evaluation, a substantial improvement over random behavior. The learned Q-table, training curves, and evaluation report are saved for reuse.

9. Future Improvements

Potential extensions include training on the harder 8x8 FrozenLake map, tuning hyperparameters (learning rate, discount factor, epsilon decay schedule) more systematically, comparing against alternative RL algorithms such as SARSA or Deep Q-Networks (DQN) for larger state spaces, and visualizing individual test episodes with Gymnasium's render_mode='human' or rgb_array recording.